# Configuration

Single rule: `appsettings*.json` **holds non-secrets ONLY.** Every secret lives in User Secrets (dev) / environment variables / Key Vault or AWS Secrets Manager (prod), referenced by an `*ApiKeyConfigKey` pointer.

## Non-secret settings (`AiSuite` section — template: `samples/SampleApi/appsettings.json`)

| Key | Example | Meaning |
|---|---|---|
| `ChatProvider` | `KnowledgeBase` / `Echo` / `OpenAiCompatible` | Chat adapter. Swapped per environment, no code change |
| `TranslationProvider` | `Echo` / `AzureTranslator` / `LibreTranslate` | Translation adapter |
| `OpenAiCompatible:Endpoint` | `https://api.openai.com/v1` | Azure: `{resource}.openai.azure.com/openai/v1`; local: `http://localhost:11434/v1` |
| `OpenAiCompatible:Model` | `gpt-4o-mini` | Model / deployment, swappable per environment |
| `OpenAiCompatible:MaxTokens` / `Temperature` | `512` / `0.7` | Per-call defaults |
| `OpenAiCompatible:ApiKeyConfigKey` | `AiSuite:ApiKeys:Chat` | **Pointer** to the secret, not the secret |
| `OpenAiCompatible:RequireApiKey` | `true` | `false` for keyless local endpoints |
| `AzureTranslator:Endpoint` / `Region` | `https://api.cognitive.microsofttranslator.com` / `southindia` | Region is required by Azure |
| `AzureTranslator:ApiKeyConfigKey` | `AiSuite:ApiKeys:Translator` | **Pointer** to the secret |
| `LibreTranslate:Endpoint` | `https://translate.internal` | Empty = provider stays unregistered |
| `KnowledgeBase:Entries` | `[{Match:[...], Answer:"..."}]` | Config-only FAQ; best-score wins |
| `KnowledgeBase:FallbackAnswer` | … | Reply when nothing matches |
| `ChatDefaults:SystemPrompt` | 15-section agent prompt (copy: customization.md → System prompt) | The agent's brain — sent to LLM providers as the first `system` message. `KnowledgeBase` ignores it (FAQ answers come from `KnowledgeBase:Entries`) |
| `ChatDefaults:MaxHistoryMessages` / `MaxMessageChars` | `20` / `4000` | History trim + input cap (cost/abuse control) |
| `Languages:Default` / `Supported` | `en` / `["en","ta","hi","es","fr"]` | **Allow-list** — anything else → 400 `VALIDATION_ERROR`. Never hard-code languages in code |
| `Resilience:TimeoutSeconds` / `MaxRetries` / `BaseDelayMs` | `60` / `2` / `500` | Retries transient (5xx/429/408/timeout) with exponential backoff + jitter; 4xx never retried |
| `RateLimit:Enabled` / `PermitLimit` / `WindowSeconds` | `false` / `20` / `60` | Built-in fixed-window limiter; enable in prod (+ `app.UseRateLimiter()`) |
| `Security:RequireAuthorization` | `false` | Require authenticated user on all AI endpoints; host plugs JWT/OIDC then flips on |

## Secret keys (complete list — TWO)

| # | Config key | Holds | Dev | Prod |
|---|---|---|---|---|
| 1 | `AiSuite:ApiKeys:Chat` | Chat API key (or empty when keyless) | User Secrets / `AiSuite__ApiKeys__Chat` | Key Vault / env / Managed Identity (no key at all) |
| 2 | `AiSuite:ApiKeys:Translator` | Translator subscription key | User Secrets / `AiSuite__ApiKeys__Translator` | Key Vault / env |

```
dotnet user-secrets init
dotnet user-secrets set "AiSuite:ApiKeys:Chat" "&lt;chat-key&gt;"
dotnet user-secrets set "AiSuite:ApiKeys:Translator" "&lt;translator-key&gt;"

// Key Vault provider (consuming project)
builder.Configuration.AddAzureKeyVault(new Uri($"https://{vault}.vault.azure.net/"), new DefaultAzureCredential());
// AWS: Amazon.Extensions.Configuration.SystemsManager / SecretsManager providers work the same way.
```

## Settings panel (`AiSuite:Settings` — runtime store)

Config seeds the defaults; the password-gated panel (demo.html , or your own admin UI on `AiSettingsService`) can change things live — no restart. Files live under the store directory (default `ai-suite-settings/`): `settings.json` (keys, voice, password hash, document index) + `kb/<id>.json` (parsed chunks).

| Key | Example | Meaning |
|---|---|---|
| `Settings:Enabled` | `true` | Expose `/settings` + `/voice` routes |
| `Settings:Password` | `pixous` | Initial password (seed only — changing it in the panel moves the source to `stored`) |
| `Settings:StoreDirectory` | `ai-suite-settings` | Relative to content root (or absolute) |
| `Settings:AllowPersistKeys` | `true` | `false` = keys stay memory-only (never touch disk) |
| `Settings:TokenHours` | `8` | Settings session lifetime |
| `Settings:MinPasswordLength` | `4` | Enforced on change |
| `Settings:ExhaustedCooldownSeconds` | `60` | 429 → short cooldown, then retry |
| `Settings:InvalidKeyCooldownHours` | `24` | 401/403 → key marked `invalid`, skipped by fallback |
| `Settings:BadConfigCooldownMinutes` | `10` | Other 4xx → config error, cooled down |

Runtime powers (all via `/api/ai/settings`, `X-Settings-Token`):

- **LLM API keys** — any vendor, multiple keys, priority order = fallback order; blanks/omitted key field on save = keep stored key; `/test` round-trips live and writes back health (ok / `invalid` / `cooling`).
- **Voice** — STT/TTS provider per side (`Browser` = free browser default, `Echo` = keyless demo, `OpenAiCompatible` = any vendor endpoint + key). Blank key on save = keep stored key.
- **Knowledge documents** — upload `.pdf` / `.docx` / `.md` / `.txt` (max 20 MB), parsed + chunked server-side; answers then cite the file.
- **Password** — panel change overrides `Settings:Password` (`passwordSource`: `config` → `stored`).

The Angular client is `AiSettingsService` (login/snapshot/keys/voice/kb); `AiVoiceService` covers runtime `stt/tts`.

## Angular configuration (no secrets — ever)

```
{ provide: AI_SUITE_CONFIG, useValue: { apiBaseUrl: 'https://api.myapp.com' } }
```

`apiBaseUrl` (your BFF backend) is the only URL the browser needs. Swap per build via Angular `environment.ts`. Verify no leakage: DevTools → Network shows only YOUR domain (`/api/ai/*`) — never `openai.com` /

`microsofttranslator.com`.